



Facultad de Ingeniería

Diseño de Aplicaciones 2

Obligatorio 1 – Evidencia del diseño y especificación de la API

“Entregado como requisito para la obtención del título de Ingeniero en Sistemas”

Repo: <https://github.com/IngSoft-DA2/271395-294598-221509.git>

Candelaria Blanco - 271395

Maximiliano Gimnenez - 294598

Ana Betina Kadessian - 221509

Tutores:

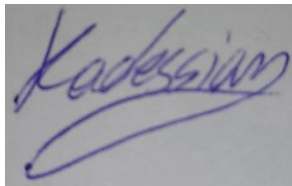
Nicolás Fierro, Maximo Healty

2025

DECLARACIÓN DE AUTORÍA

Nosotros, Candelaria Blanco, Maximiliano Gimenez y Betina Kadessian, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el obligatorio 1 de la materia Diseño de Aplicaciones 2.
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra
- En la obra, hemos acusado recibo de las ayudas recibidas
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Ana Betina Kadessian

8/5/2025



Candelaria Blanco

8/5/2025



Maximiliano Gimenez

8/5/2025

ABSTRACT

Este trabajo presenta el desarrollo de un Simulador de Objetos que permite modelar y ejecutar estructuras propias de la Programación Orientada a Objetos (POO). El sistema posibilita la creación de clases con herencia simple, definición de atributos y métodos, y ejecución de métodos respetando accesibilidad, herencia y polimorfismo. También permite simular patrones de diseño como Template Method y Strategy. La solución fue implementada como una API REST utilizando .NET 8 y Entity Framework Core, siguiendo buenas prácticas de diseño, Clean Code y TDD. Este proyecto fue desarrollado para la Universidad ORT Uruguay como parte de la materia Diseño de Aplicaciones 2, con el objetivo de aplicar conceptos avanzados de diseño y arquitectura de software en un entorno práctico y educativo.

Índice

Discusión de los criterios seguidos para asegurar que la API cumple con los criterios REST.	4
Criterios REST	4
Buenas prácticas seguidas en endpoints	4
Descripción del mecanismo de autenticación de requests.	5
Descripción general de códigos de respuesta	5
1xx	5
2xx	5
3xx	6
4xx	6
5xx	6
Formato JSON de las respuestas	6
Respuestas de error	6
Respuestas de éxito	6
Descripción de los resources de la API.	7
URL base.	7
Descripción de cada resource	7

Discusión de los criterios seguidos para asegurar que la API cumple con los criterios REST.

Para garantizar que la API se adhiera a los principios REST, se implementan los lineamientos detallados a continuación.

Criterios REST

1. Sin estado
Cada solicitud enviada por el cliente debe contener toda la información necesaria para que el servidor pueda interpretarla y procesarla correctamente. Nos aseguramos de que el servidor no mantenga el estado del cliente entre solicitudes consecutivas.
2. Arquitectura Cliente-Servidor
El cliente se encarga de mostrar la interfaz y recopilar la información que el usuario ingresa en cada solicitud. Mientras que el servidor se encarga de procesar esas solicitudes, acceder a los datos necesarios, aplicar lógica de negocio, y devolver una respuesta adecuada al cliente.
3. Interfaz Uniforme
Los recursos son identificados mediante URIs con nombres en plural, siguiendo las buenas prácticas y principios REST (detallados a continuación) para mantener una interfaz coherente. La API emplea los métodos estándar del protocolo HTTP (GET, POST, PUT, DELETE) junto con códigos de estado apropiados.
4. Sistema en Capas
La arquitectura de la API permite el uso de una estructura por capas, facilitando la integración de intermediarios de carga para mejorar tanto la escalabilidad como la seguridad del sistema.
5. Cacheabilidad
Para optimizar el rendimiento, se definieron explícitamente qué respuestas son cacheables y cuáles no, alineándose con los principios REST.

Buenas prácticas seguidas en endpoints

1. Sustantivos ante verbos
Incorrecto: `../create-classes`
Correcto: `POST ../classes`, el verbo determina y hace explícita la operación.
2. Intuitiva y simple
Si mediante la URI se puede entender lo que hace la API sin necesitar ningún tipo de documentación extra, será más simple de utilizarla.
Dentro de este punto tratamos determinar nombres concretos ante abstractos.

3. Plural ante singular

Para procurar la uniformidad del sistema elegimos uno de los formatos y todos las operaciones deberán respetarlo. Preferentemente plural para mejor entendimiento.

Correcto: *../dogs, ../admins*

Evitar: *../things, ../someone*

4. En minúscula

5. Nunca tener dos recursos seguidos

Incorrecto: *../recources1/resources2*

Correcto: *../recources1/{id}/resources2*

Descripción del mecanismo de autenticación de requests.

Para esta entrega no se pide ningún tipo de autenticación, por lo tanto no fue implementada.

Descripción general de códigos de respuesta

1xx

Códigos informativos: indican que el servidor ha recibido la solicitud y que se está procesando, aunque aún no hay una respuesta definitiva disponible. Le informan al cliente que debe esperar una respuesta más completa o continuar enviando datos si la solicitud es extensa.

Esta clasificación de códigos no fue aplicados en este obligatorio.

2xx

Códigos de éxito: confirman que la solicitud fue recibida, comprendida y ejecutada correctamente por el servidor. Indican que la operación se completó con éxito.

- 200 OK: utilizado para indicar que la solicitud se ha procesado con éxito. Puede aplicarse a cualquier otra acción solicitada por el cliente. No necesariamente significa que el contenido haya cambiado, solo que la operación solicitada fue exitosa.
- 201 Created: se utiliza específicamente cuando se crea un nuevo recurso como resultado de la solicitud (operación POST).
- 204 No Content: se utiliza específicamente para una eliminación de recursos exitosas.

3xx

Códigos de redirección: informan que la solicitud no se puede completar en su forma actual, pero que el cliente debe realizar una acción adicional, como seguir una URL diferente o reenviar la solicitud a otro punto.

Esta clasificación de códigos no fue aplicados en este obligatorio.

4xx

Códigos de errores del cliente: indican que hubo un problema con la solicitud enviada por el cliente. Puede deberse a errores de sintaxis, datos inválidos, falta de autenticación o intento de acceder a recursos inexistentes o restringidos.

- 400 Bad Request: utilizado para errores de validación y operaciones inválidas. EJ: crear una clase sin indicar su nombre.
- 404 Not Found: ocurre cuando el servidor no puede encontrar el recurso solicitado. EJ: consultar una clase que no existe.
- 409 Conflict: se utiliza para cuando el cliente desea realizar operaciones no idempotente sobre recursos vinculados a otros existentes (en síntesis se utiliza en conflictos de recursos en uso). EJ: se desea eliminar un método que es invocado por otros métodos.

5xx

Códigos de errores del servidor: reflejan fallos en el servidor al procesar una solicitud válida. Aunque el cliente hizo todo correctamente, un problema interno impidió completar la operación.

- 500 Internal Server Error: Para excepciones no manejadas

Formato JSON de las respuestas

Respuestas de error

```
{  
  InnerCode: int,  
  Message: string  
}
```

Respuestas de éxito

- Creación y modificación:

```
{  
  Message: string (idóneo para cada caso),  
  TipoObjetoManipulado:
```

- ```

 [lista de información de cada atributo del objeto]
 }

```
- Eliminación: *body response* vacío.
  - Obtención de la información (GET):
 

```

 {
 [lista de información de cada atributo del objeto]
 }

```

## Descripción de los resources de la API.

### URL base.

Dominio: <http://localhost:5128/api/v1/>

### Descripción de cada resource

Para ello generamos la siguiente tabla (pág. 8)

| DESCRIPCIÓN                                                                                              | RECURSO     | VERBO  | URI                               | HEADER       | BODY                                                                                                                       | RESPONSE                                                                                                                                                                                                   | ERRORES                                                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------|-------------|--------|-----------------------------------|--------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Crear una nueva clase con sus datos pertinentes (pero sin los métodos ni atributos, luego se incluirán). | Clases      | POST   | /api/v1/classes                   | location     | Name: string, BaseClassId: guid, State: {"Normal","Sealed","Abstract"},                                                    | 201 Created. Message: "Clase creada exitosamente", SimClass:{Name: string, BaseClassId: guid, State: {"Normal","Sealed","Abstract"}, Methods: [], Attributes: []}                                          | 400 Bad Request: nombre, estado o estructura del JSON inválidos, 404 Not Found: no existe en la clase base, 409 Conflict: nombre de clase en uso                                                                                                                |
| Listar todas las clases registradas                                                                      | Clases      | GET    | /api/v1/classes                   | Content-Type | N/A                                                                                                                        | 200 Ok. {[Id: guid, Name: string, BaseClassId: guid, State: {"Normal","Sealed","Abstract"}, Methods: [], Attributes: []]}                                                                                  |                                                                                                                                                                                                                                                                 |
| Eliminar una clase                                                                                       | Clases      | DELETE | /api/v1/classes/{id}              | Content-Type | N/A                                                                                                                        | 204 No Content.                                                                                                                                                                                            | 404 Not Found: la clase no existe, 409 Conflicto: la clase está usada no se puede eliminar                                                                                                                                                                      |
| Actualizar una clase                                                                                     | Clases      | PUT    | /api/v1/classes/{id}              | Content-Type | Id: guid, Name: string, BaseClassId: guid, State: {"Normal","Sealed","Abstract"}, Methods: [], Attributes: []              | 200 OK: Message: "Clase actualizada exitosamente", SimClass:{Name: string, BaseClassId: guid, State: {"Normal","Sealed","Abstract"}, Methods: [], Attributes: []}                                          | 400 Bad Request: atributos inválidos o problemas con metodos/atributos (ej: métodos abstractos sin implementar), 404 Not Found: la clase o alguno de sus atributos no existen. 409 Conflict: está siendo usada por otras siendo entidades, no se puede eliminar |
| Obtener la información de una clase                                                                      | Clases      | GET    | /api/v1/classes/{Id}              | Content-Type | N/A                                                                                                                        | 200 Ok. {Id: guid, Name: string, BaseClassId: guid, State: {"Normal","Sealed","Abstract"}, Methods: [], Attributes: []}                                                                                    | 404 Not Found: la clase no existe                                                                                                                                                                                                                               |
| Crear un método de una clase                                                                             | Métodos     | POST   | /api/v1/classes/{Id}/methods      | location     | Name: string, Privacy: {"Public","Private","Protected"}, Accesibilidad: {"Normal","Sealed","Abstract"}, IdReturnType: guid | 201 Created. Message: "Método creado exitosamente", SimMethod:{Name: string, Privacy: {"Public","Private","Protected"}, Accesibilidad: {"Normal","Sealed","Abstract"}, IdReturnType: guid, Parameters: []} | 400 Bad Request: los datos del método no son válidos o no cumplen con la lógica de negocios, 404 Not Found: algún atributo de la información del método no existe                                                                                               |
| Listar todos los métodos de una clase                                                                    | Métodos     | GET    | /api/v1/classes/{ClassId}/methods | Content-Type | N/A                                                                                                                        | 200 Ok. {[Id: guid, Name: string, Privacy: {"Public","Private","Protected"}, Accesibilidad: {"Normal","Sealed","Abstract"}, IdReturnType: guid, Parameters: []]}                                           |                                                                                                                                                                                                                                                                 |
| Mostrar un método específico                                                                             | Métodos     | GET    | /api/v1/methods/{id}              | Content-Type | N/A                                                                                                                        | 200 Ok. {Id: guid, Name: string, Privacy: {"Public","Private","Protected"}, Accesibilidad: {"Normal","Sealed","Abstract"}, IdReturnType: guid, Parameters: []}                                             | 404 Not Found: No existe método con este id                                                                                                                                                                                                                     |
| Eliminar un método de una clase                                                                          | Métodos     | DELETE | /api/v1/methods/{id}              | Content-Type | N/A                                                                                                                        | 204 Not content                                                                                                                                                                                            | 404 Not Found: No existe método con este id                                                                                                                                                                                                                     |
| Agregar parámetro de un método                                                                           | Métodos     | POST   | /api/v1/methods/{id}/parameters   | location     | Name: string, ClassTypeId: string                                                                                          | 201 Created. Message: "Parámetro creado exitosamente", Parametro: {Id: guid, Name: string, MethodId: string, ClassTypeId: string}                                                                          | 400 Bad Request: formato incorrecto o no cumple con la lógica de negocio, 404 Not Found: metodo asociado o el tipo no existe                                                                                                                                    |
| Agregar variables local de un método                                                                     | Métodos     | POST   | /api/v1/methods/{id}/variables    | location     | Name: string, ClassTypeId: string                                                                                          | 201 Created. Message: "Variable creada exitosamente", Variable: {Id: string, Name: string, MethodId: string, ClassTypeId: string}                                                                          | 400 Bad Request: formato incorrecto o no cumple con la lógica de negocio, 404 Not Found: metodo asociado o el tipo no existe                                                                                                                                    |
| Agregar método que invoca un método                                                                      | Métodos     | POST   | /api/v1/methods/{id}/invocations  | location     | IdReference: guid, MethodName: string, Parameters: [], TypeReference: guid                                                 | 201 Created. Message: "Invocación creada exitosamente", Invocacion:{Id: guid, idReference:guid, typeReference: guid, methodName: string, parameter: []}                                                    | 400 Bad Request: algún atributo de la información no existe, 404 Not Found: cuando las referencias no existen (método, clase, parámetro, etc.)                                                                                                                  |
| Ver información de un parámetro de un método específico                                                  | Métodos     | GET    | /api/v1/parameters/{id}           | Content-Type | N/A                                                                                                                        | 200 OK. Id:guid, Name: string, ClassId: guid, MethodId: guid, ClassName: string, Type: string                                                                                                              | 404 Not Found: el parámetro no existe                                                                                                                                                                                                                           |
| Ver información de una variable de un método específico                                                  | Métodos     | GET    | /api/v1/variables/{id}            | Content-Type | N/A                                                                                                                        | 200 OK. Id: guid, MethodId: guid, Name: string, ClassId: guid, ClassName: string, Type: string                                                                                                             | 404 Not Found: la variable no existe                                                                                                                                                                                                                            |
| Ver información de una invocación de un método específico                                                | Métodos     | GET    | /api/v1/invocations/{id}          | Content-Type | N/A                                                                                                                        | 200 OK. Id: guid, MethodId: guid Signature: {MethodId:guid, Parameters: []}, Reference: {Type: guid, VariableId: guid, ClassName: string},                                                                 | 404 Not Found: la invocación no existe                                                                                                                                                                                                                          |
| Crear un atributo de una clase                                                                           | Atributos   | POST   | /api/v1/classes/{id}/attributes   | location     | Name: string, IdClassType: guid, Privacy: {"Public","Private","Protected"}                                                 | 201 Created. Message: "Atributo creado exitosamente", Atributo: {Id: guid, Name: string, Typeld: guid, Privacy:{"Public","Private","Protected"}, RelatedClassId: guid}                                     | 404 Not found: No existe clase con ese id                                                                                                                                                                                                                       |
| Eliminar un atributo de una clase                                                                        | Atributos   | DELETE | /api/v1/attributes/{id}           | Content-Type | N/A                                                                                                                        | 204 Sin contenido                                                                                                                                                                                          | 404 Not Found: No existe ningun atributo con esta id en la clase                                                                                                                                                                                                |
| Modificar un atributo de una clase                                                                       | Atributos   | PUT    | /api/v1/attributes                | Content-Type | Id, guid, Name: string, Typeld: guid, Privacy: {"Public","Private","Protected"}, RelatedClassId: guid                      | 200 OK. Id: guid, Name: string, CalueClassId: guid, ClassName: string, Visibility: {"Protected","Private","Public"}                                                                                        | 404 Not Found: El atributo no existe en esta clase, 400 Bad Request: Campos inválidos                                                                                                                                                                           |
| Ejecutar un método                                                                                       | Ejecuciones | POST   | /api/v1/executions                | location     | MethodName: string, Parameters: [], IdReferenceType: guid, IdInstanceType: guid                                            | 200 OK: string de la ejecución                                                                                                                                                                             | 400 Bad Request: problemas de accesibilidad o ejecución inválida. 404 Not Found: metodo o sus atributos no existen                                                                                                                                              |